MockKernel

System Architecture & Product Requirements Document (PRD)

Sprint Duration: 15 Days | Team Size: 6 Engineers

1. Executive Summary

This document serves as the comprehensive blueprint for building MockKernel, an interactive coding environment featuring real-time code execution, adaptive AI interviews, and live contest capabilities. The architecture leverages a unified JavaScript/TypeScript ecosystem across both frontend and backend, heavily utilizing Docker for containerized workloads, ensuring a smooth and consistent development experience for the team.

2. Product Requirements (PRD)

a. Core Product Features

- **Real-time Code Execution:** Integrated Monaco editor with an asynchronous code evaluation engine and live console/verdict panel.
- **Adaptive AI Interviews:** Voice-enabled AI interactions focusing on general CS fundamentals (OS, DBMS, Data Structures). Uses Web Speech API and Groq (Llama-3.3).
- **Contest Engine & Dashboards:** Dedicated interfaces for students and administrators to manage problems (CRUD) and participate in live assessments.
- **Live Plagiarism Detection:** AST tokenization and Levenshtein distance matching, isolated to the scope of active live contests to optimize performance.
- **Real-time Leaderboard:** High-performance ranking system powered by Redis (ZINCRBY).

b. User Journeys

- **The Competitor:** Logs in, enters a contest, writes code in the Monaco editor, and submits. Socket.io streams the live status (Pending → Completed/TLE) directly to their verdict panel.

- **The Interviewee:** Initiates an AI interview. Answers are spoken out loud, captured via Web Speech API, and evaluated conceptually by the AI, which replies dynamically in the chat UI.
- **The Admin/Event Organizer:** Uses the dashboard to configure coding problems, monitor live submissions, and oversee contest operations and participant metrics.

c. Functional Requirements

- Code submissions must be offloaded from the main Express API to a BullMQ worker queue.
- Real-time bidirectional communication must be maintained via Socket.io for verdicts and updates.
- AI prompt logic must utilize pre-defined templates for CS fundamentals and return data in strictly structured JSON format.
- API rate limiting must be enforced at the gateway utilizing a Redis Token-Bucket algorithm.

d. Acceptance Criteria

- Code execution verdicts (Accepted, Wrong Answer, TLE) must be delivered within 10 seconds.
- The live leaderboard must reflect score changes instantly upon a successful, non-plagiarized submission.
- A seamless fallback to manual text input must activate if microphone access is denied or unsupported.

3. System Architecture & Workflows

The system is strictly decoupled into six operational zones to ensure scalability and ease of local development:

- **ZONE A: Client Environment** - React (Vite) + TS + Tailwind CSS. Hosts the Monaco Editor, Web Speech API, and dashboards.
- **ZONE B: API Gateway** - Node.js + Express + TS. Handles Auth (JWT), CRUD, Plagiarism AST logic, and the Adaptive AI controller.
- **ZONE C: State & Persistence** - MongoDB Atlas for primary records (Users, Submissions) and Redis for fast transient state (Queues, Rate Limiting, Leaderboards).
- **ZONE D: Async Worker Pool** - Dedicated Node.js BullMQ consumers handling heavy submission jobs and webhooks.

- **ZONE E: Execution Sandbox** - Judge0 CE running in isolated Docker cgroups to safely compile untrusted code.
- **ZONE F: External Services** - Groq API (Llama-3.3) for blazing-fast AI inference.

> **Edge Cases & Fallbacks:**
> - **E1 (Hardware Failure):** Fallback to standard textarea if Web Speech API fails.
> - **E2 (Infinite Loops):** 10-second Timeout Guard on workers terminates hanging Judge0 tasks, marking them as TLE.
> - **E3 (API Limits):** Exponential backoff and retry queue if Groq API returns 429 Too Many Requests.
> - **E4 (Spam Submissions):** Redis blocks excessive "Run Code" clicks at the gateway layer.

4. Infrastructure & Implementation Strategy

Judge0 Execution Environment

To keep costs strictly at zero without compromising the performance of containerized compilation, avoid installing Judge0 on the primary API server. Instead, utilize the **GitHub Student Developer Pack** to provision a free, robust Ubuntu droplet (e.g., via DigitalOcean credits). Deploy the Judge0 `docker-compose.yml` on this dedicated instance. Alternatively, for completely localized testing, run the Docker stack on a dedicated campus machine and expose it securely via Ngrok/Cloudflare Tunnels.

Tech Stack & Standardization

To maximize the team's velocity during the 15-day sprint, the stack remains strictly within the Node ecosystem:

`React (Vite)` `TypeScript` `Zustand (State)` `Node.js + Express` `MongoDB + Mongoose` `Redis` `BullMQ` `Docker`

5. CI/CD & Team Collaboration Protocol

With 6 engineers working simultaneously, strict workflow rules are required:

- **Local Development:** A master `docker-compose.yml` will be maintained. Every engineer can spin up the required MongoDB, Redis, and BullMQ environment instantly using `docker compose up`.

- **API Contracts First:** Before UI components or backend controllers are written, exact JSON payload structures must be defined. This allows frontend and backend work to proceed in parallel.
- **Branching Strategy:** Direct commits to `main` are prohibited. All work must happen on feature branches (e.g., `feat/auth`, `fix/socket-drop`).
- **Continuous Integration (CI):** GitHub Actions will be configured to automatically run `npm install`, ESLint, and TypeScript type-checking on every Pull Request. A PR cannot be merged unless it passes these automated checks and receives at least one peer review.
- **Continuous Deployment (CD):** Upon merging to `main`, the frontend will automatically deploy via Vercel, and the Express backend will deploy via a platform like Render or Railway.